

1. Сборка симулятора из исходных текстов

1.1 Сборка в ОС Windows

Симулятор ориентирован на использование в 64-разрядных версиях операционной системы Windows. Перед началом сборки игры следует убедиться в том, что

- У вас установлена актуальная версия ОС **со всеми необходимыми для работы обновлениями**
- Установлена **актуальная версия драйвера видеокарты, поддерживающая Vulkan API версии не ниже 1.1.170**

Для сборки проекта симулятора требуется установленный фреймворк Qt6 (рекомендуемая версия Qt 6.8.0), собранный 64-разрядным компилятором MinGW (рекомендуемая версия MinGW 13.1.0 64-bit).

Кроме того, для сборки проекта требуется система сборки CMake версии не ниже 3.29.

В качестве зависимостей для сборки компонентов графической библиотеки VulkanSceneGraph требуется установка Lunar G Vulkan SDK для 64-разрядной ОС Windows.

1.1.1 Зависимости сборки

Симулятор использует ряд сторонних библиотек. Все зависимости собраны из исходных текстов компилятором MinGW. Для упрощения процесса подготовки окружения можно воспользоваться прекомпилированными версиями зависимостей, список которых, вместе со ссылками на скачивание, приведен ниже

Наименование	Версия	Назначение	Загрузка
VulkanSceneGraph, vsgXchange, vsgImGui	1.1.15	Графический фреймворк для разработки 3D-графики на базе Vulkan API	СКАЧАТЬ
GLSLang	15.1.0	Компилятор шейдеров GLSL и инструменты для работы с шейдерами SPIR-V	СКАЧАТЬ
openal-soft	1.23.1	Кроссплатформенный API для работы с аудиоданными	СКАЧАТЬ
FreeType	2.13.3	Библиотека для растеризации шрифтов и операций над ними	СКАЧАТЬ
assimp	5.4.3	Кроссплатформенная библиотека импорта 3D-моделей, обеспечивающая общий интерфейс программирования приложений (API) для различных форматов.	СКАЧАТЬ
SFML	3.0.0	Мультимедийная библиотека, обеспечивающая поддержку джойстиков в симуляторе	СКАЧАТЬ
Lua	5.4.8	Интерпретатор скриптового языка Lua	СКАЧАТЬ
sol2	3.3.0	Библиотека интеграции Lua в приложения на C++	СКАЧАТЬ
KTX-Software	4.4.2	Библиотеки и инструменты для работы с текстурами формата KTX	СКАЧАТЬ

Внимание! Содержимое каждого из приведенных в списке 7-zip архивов распаковывается в отдельный каталог в произвольном, удобном вам месте жесткого диска. Для распаковки архивов требуется установка архиватора 7-zip, получить который можно [здесь](#)

1.1.2 Настройка окружения сборки

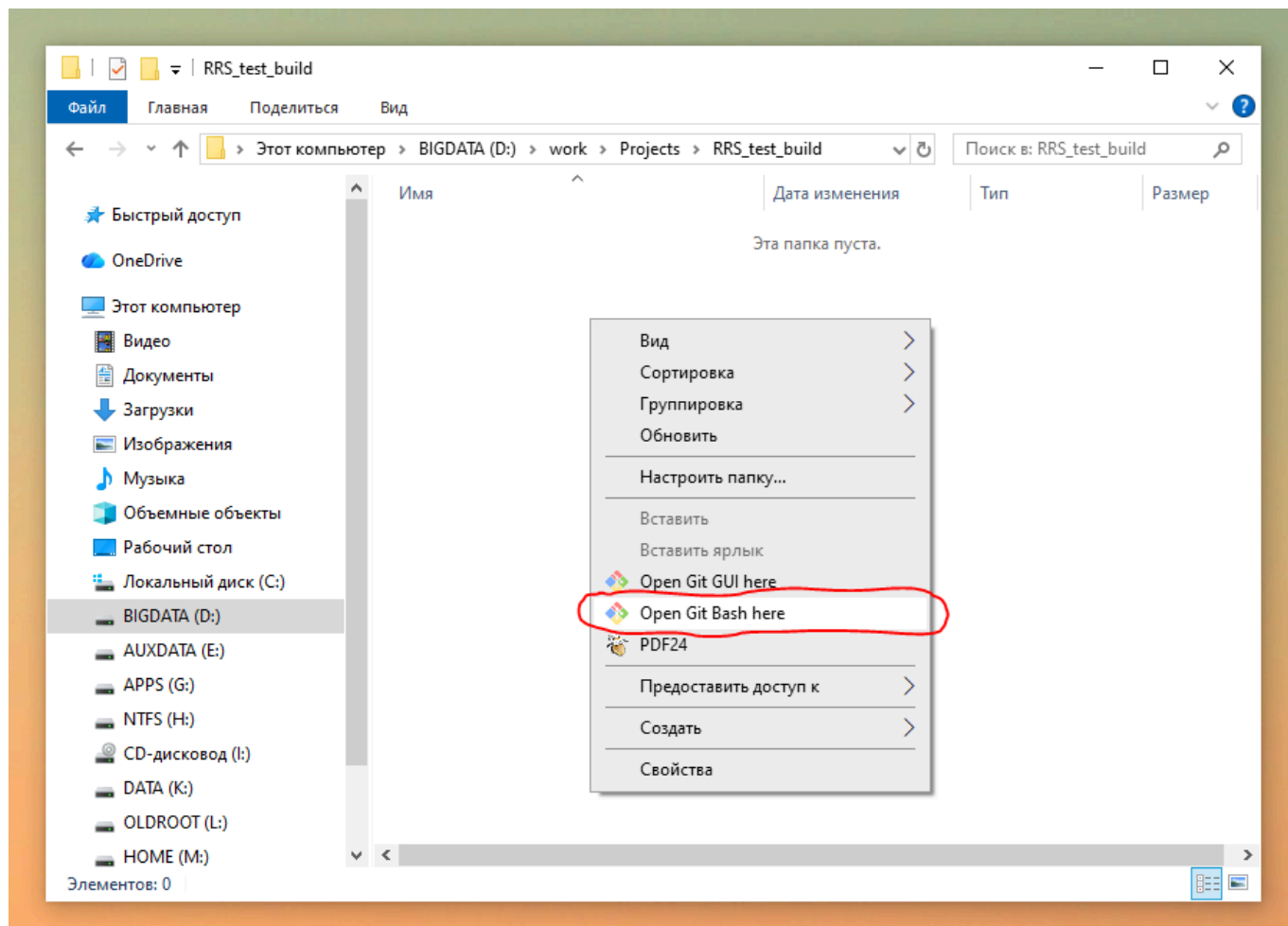
После установки вышеприведенных зависимостей **необходимо настроить следующие системные переменные окружения**

Имя переменной	Значение переменной
OPENAL_BIN	содержит путь к каталогу <code>bin/</code> библиотеки OpenAL
OpenAL_DIR	содержит путь к каталогу <code>lib/cmake/OpenAL</code> библиотеки OpenAL
FREETYPE_DIR	Содержит путь к корневому каталогу библиотеки FreeType. Обратите на это внимание, чтобы избежать ошибок настройки сборки!
vsg_DIR	содержит путь к каталогу <code>lib/cmake/vsg</code> библиотеки VulkanSceneGraph
vsgXchange_DIR	содержит путь к каталогу <code>lib/cmake/vsgXchange</code> библиотеки vsgXchange
vsgImGui_DIR	содержит путь к каталогу <code>lib/cmake/vsgImGui</code> библиотеки vsgImGui
glslang_DIR	содержит путь к каталогу <code>lib/cmake/glslang</code> библиотеки GLSlang
SPIRV-Tools_DIR	содержит путь к каталогу <code>lib/cmake/SPIRV-Tools</code> библиотеки SPIRV-Tools
SPIRV-Tools_DIR-opt	содержит путь к каталогу <code>lib/cmake/SPIRV-Tools-opt</code> библиотеки SPIRV-Tools-opt
assimp_DIR	содержит путь к каталогу <code>lib/cmake/assimp-5.4</code> библиотеки assimp
SFML_DIR	содержит путь к каталогу <code>lib/cmake/SFML</code> библиотеки SFML
lua_DIR	содержит путь к каталогу <code>lib/cmake/lua</code> библиотеки Lua
sol2_DIR	содержит путь к каталогу <code>lib/cmake/sol2</code> библиотеки sol2
Ktx_DIR	содержит путь к каталогу <code>lib/cmake/ktx</code> библиотеки KTX-Software

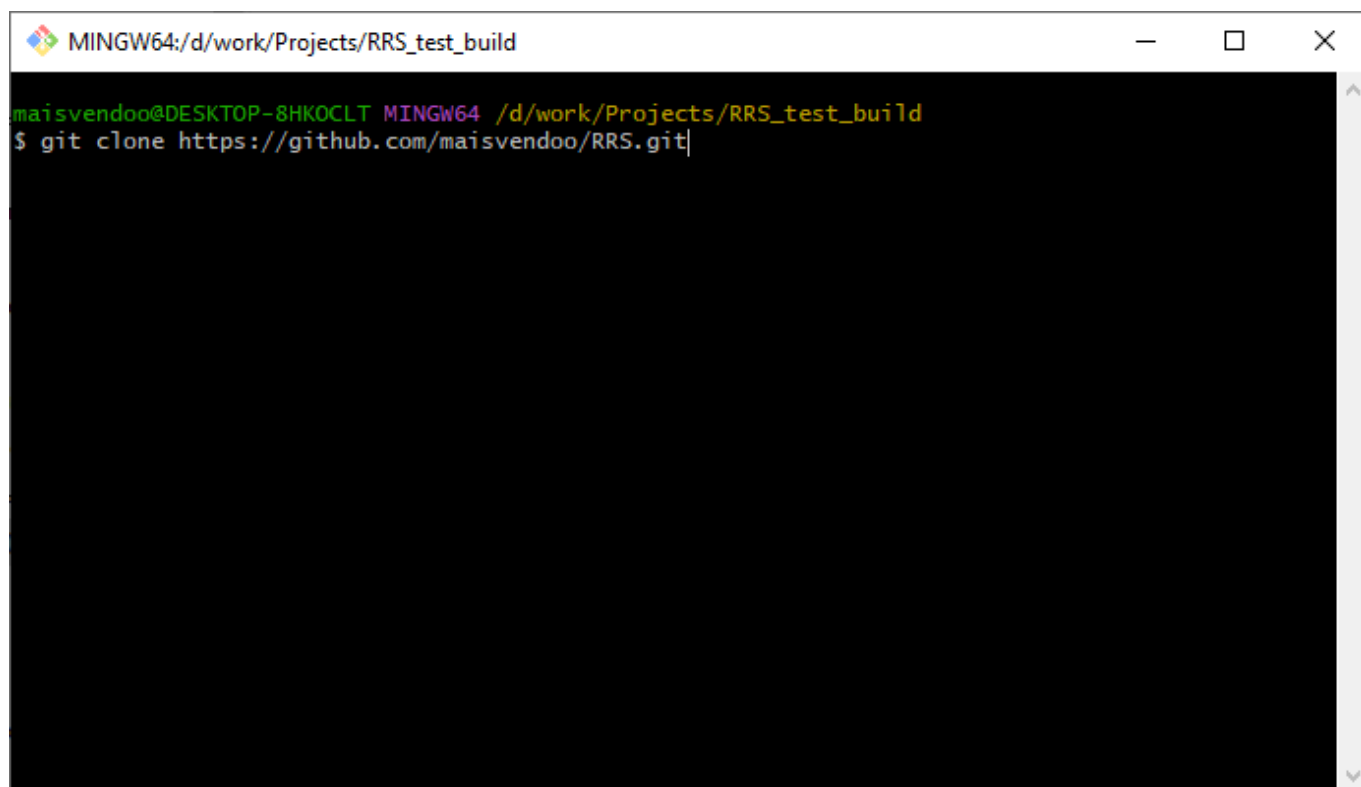
1.1.3 Получение исходного кода симулятора

Исходный код симулятора размещен на ресурсе Github. при работе над проектом используется система контроля версий Git, в связи с чем, для корректной работы с деревом проекта следует установить инструментарий для работы с Git.

В удобном для вас каталоге вызываем консоль GitBash

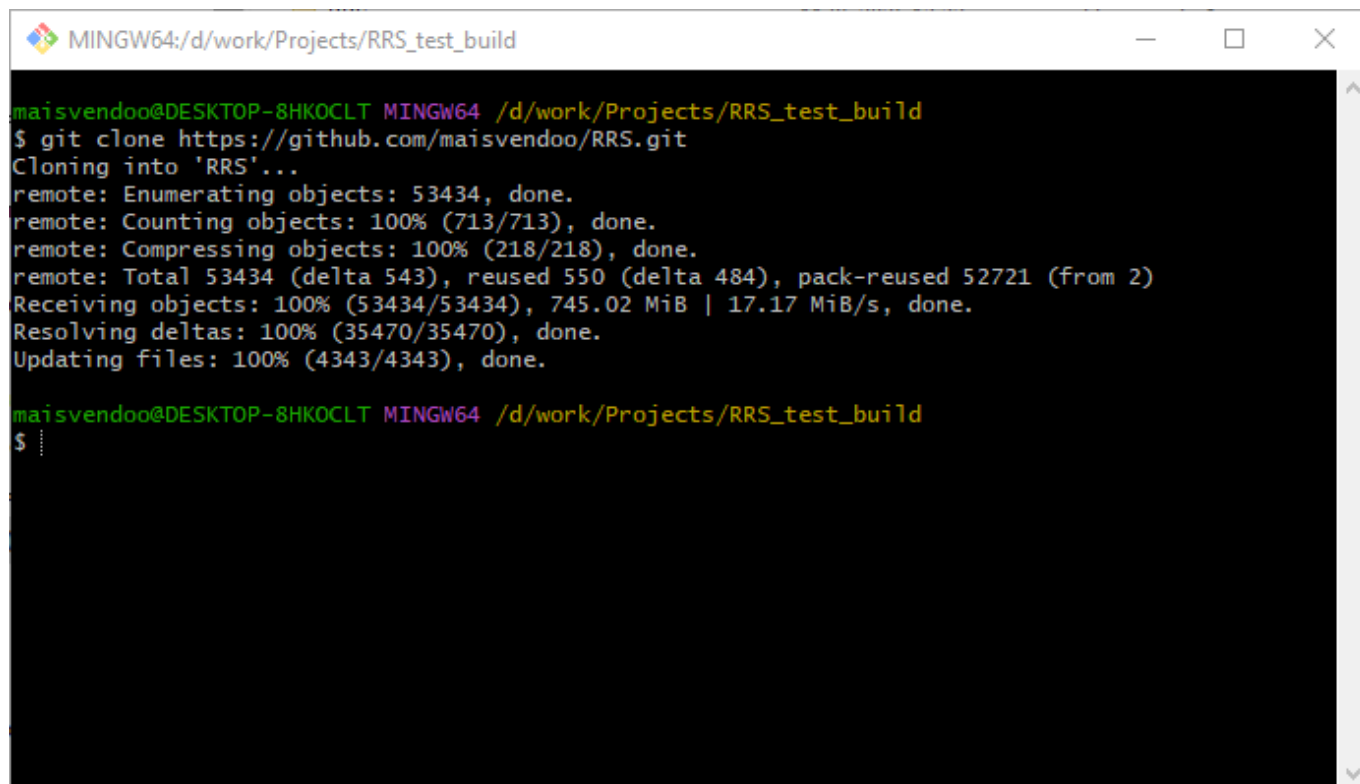


В появившемся терминале вводим команду



ВНИМАНИЕ! Если вы создали форк репозитория RRS в своем аккаунте Github — в качестве ссылки на репозиторий в данной команде укажите ссылку на ваш репозиторий! Данное действие необходимо для коллективной работы над проектом и публикации своих изменений, с целью последующего добавления их в основной репозиторий проекта посредством pull-requests.

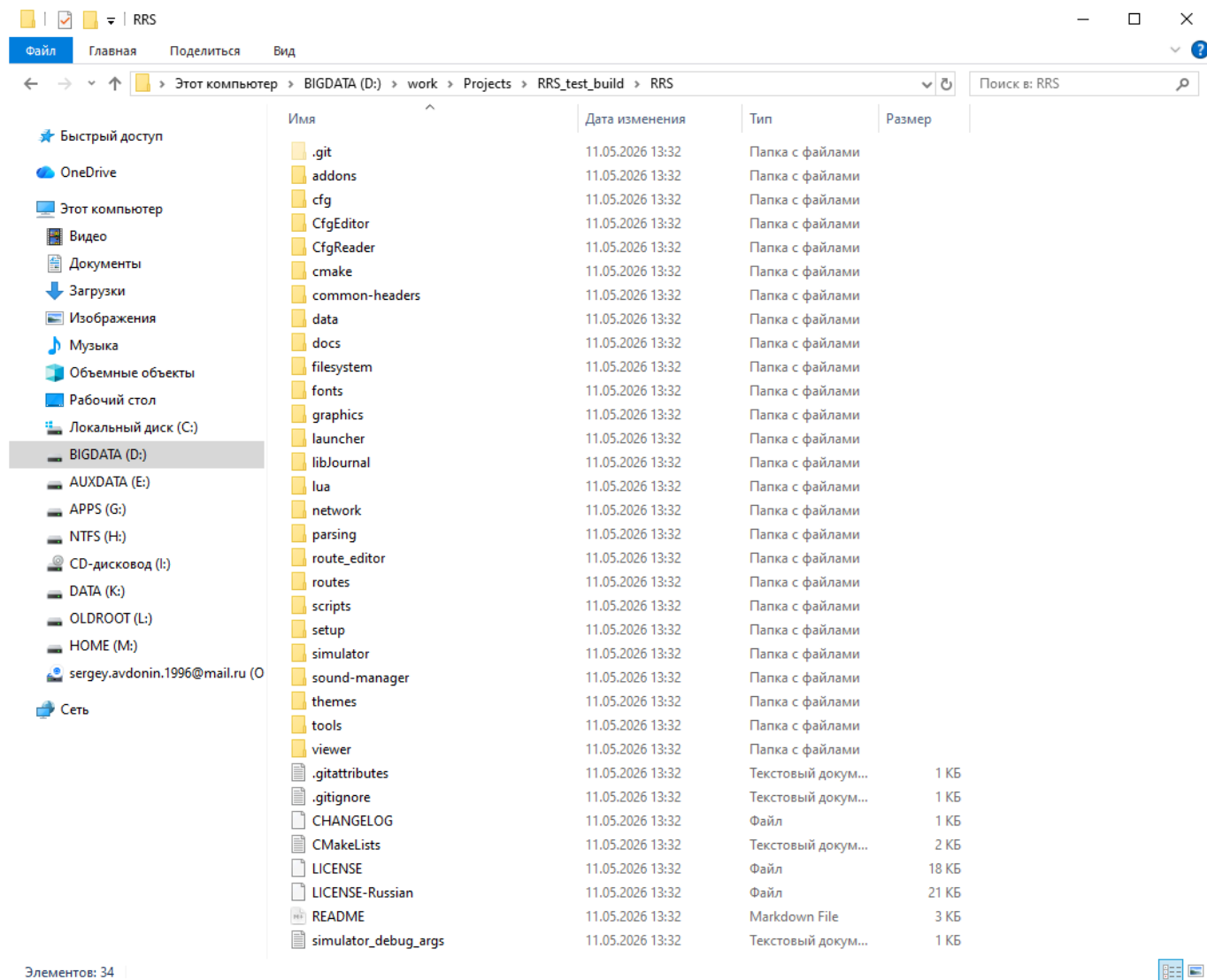
Начнется процесс клонирования репозитория на вашу локальную систему, который успешно завершается так

A screenshot of a terminal window titled 'MINGW64:/d/work/Projects/RRS_test_build'. The terminal shows the execution of the 'git clone' command to clone the repository 'RRS' from GitHub. The output displays the progress of cloning, including object enumeration, counting, compressing, and downloading. The process completes successfully, and the terminal returns to the command prompt.

```
maisivendoo@DESKTOP-8HKOCLT MINGW64 /d/work/Projects/RRS_test_build
$ git clone https://github.com/maisivendoo/RRS.git
Cloning into 'RRS'...
remote: Enumerating objects: 53434, done.
remote: Counting objects: 100% (713/713), done.
remote: Compressing objects: 100% (218/218), done.
remote: Total 53434 (delta 543), reused 550 (delta 484), pack-reused 52721 (from 2)
Receiving objects: 100% (53434/53434), 745.02 MiB | 17.17 MiB/s, done.
Resolving deltas: 100% (35470/35470), done.
Updating files: 100% (4343/4343), done.

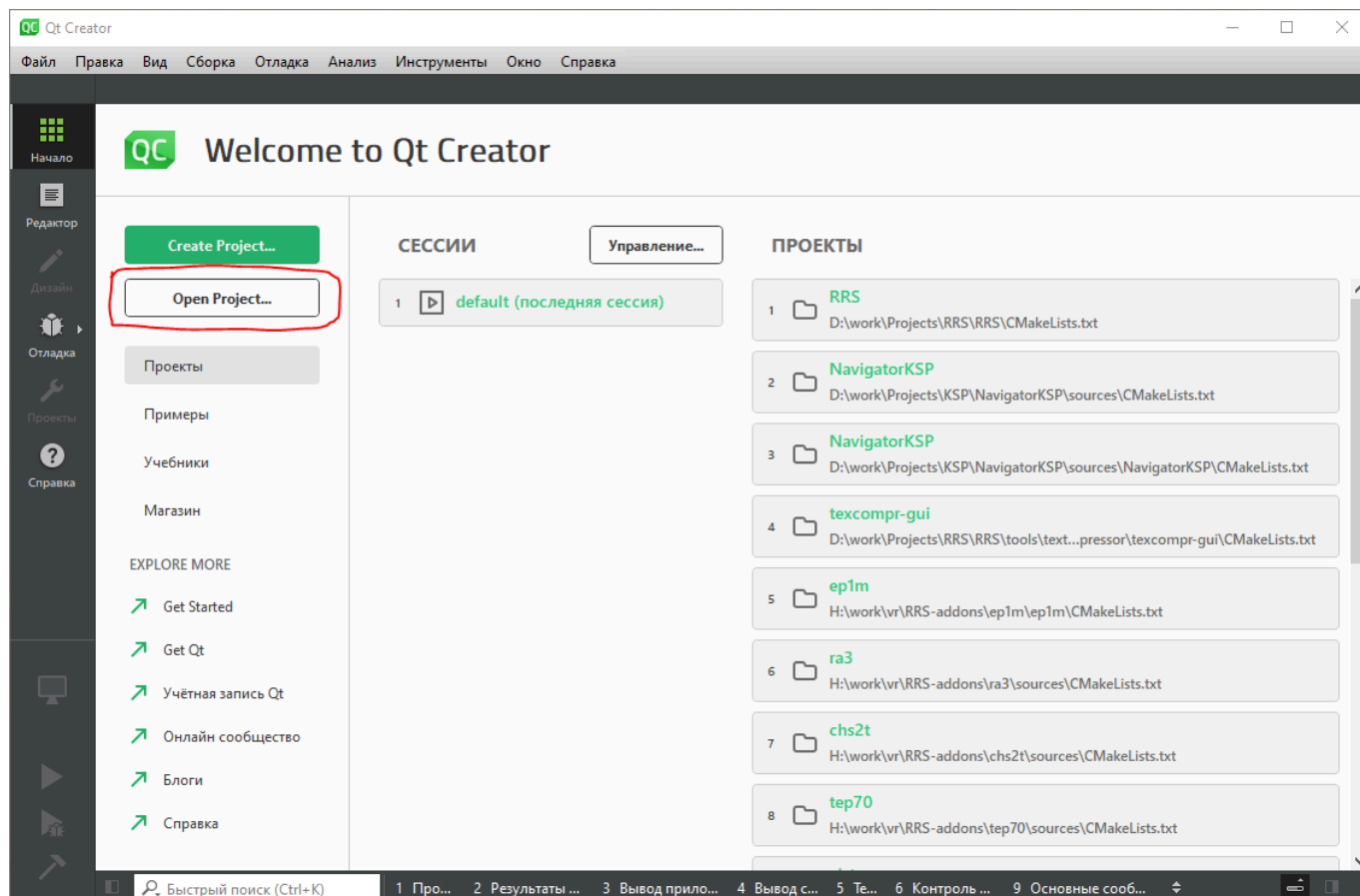
maisivendoo@DESKTOP-8HKOCLT MINGW64 /d/work/Projects/RRS_test_build
$
```

В образовавшемся каталоге у нас развернуто полное дерево исходников симулятора

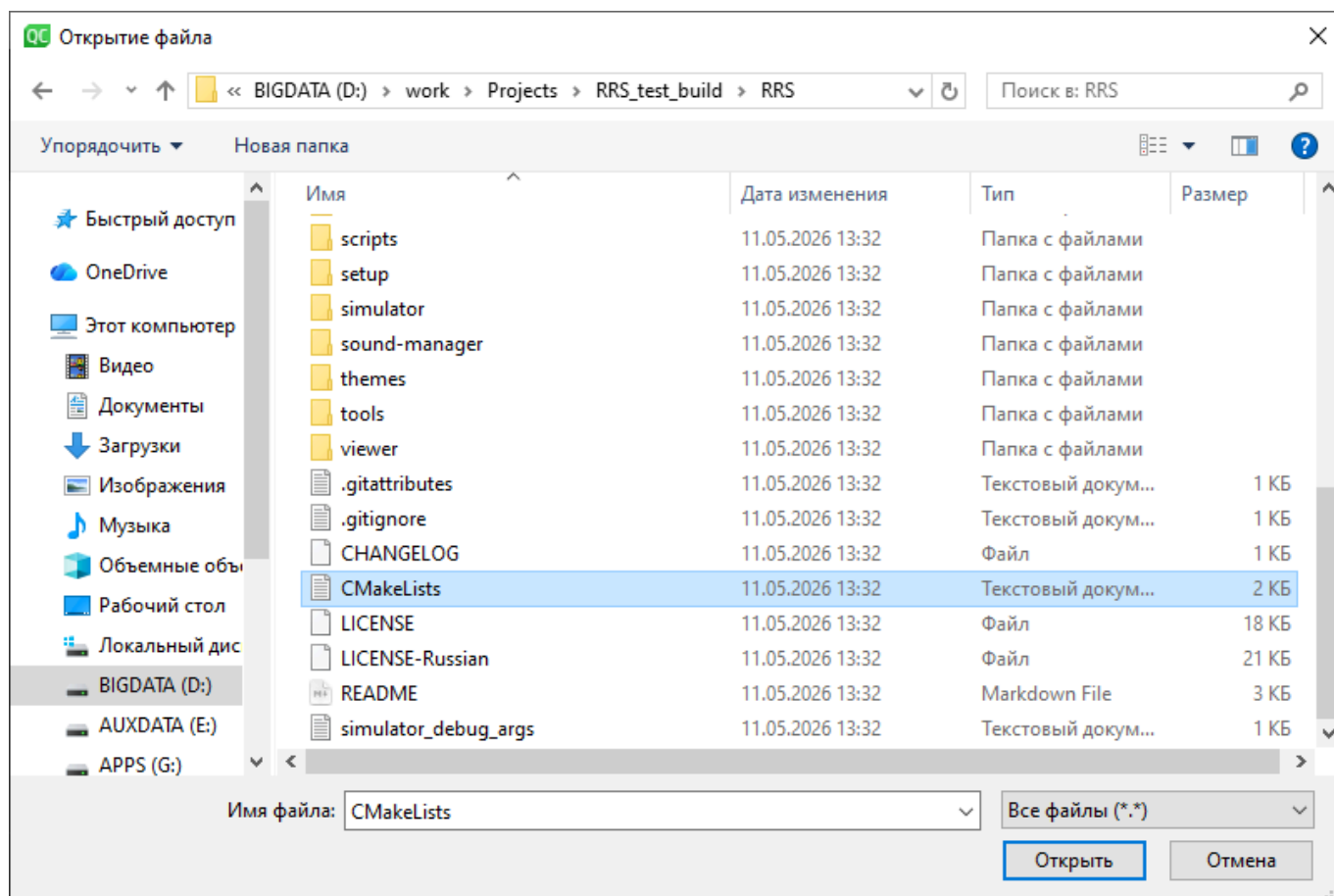


1.1.4 Настройка проекта и сборка в QtCreator

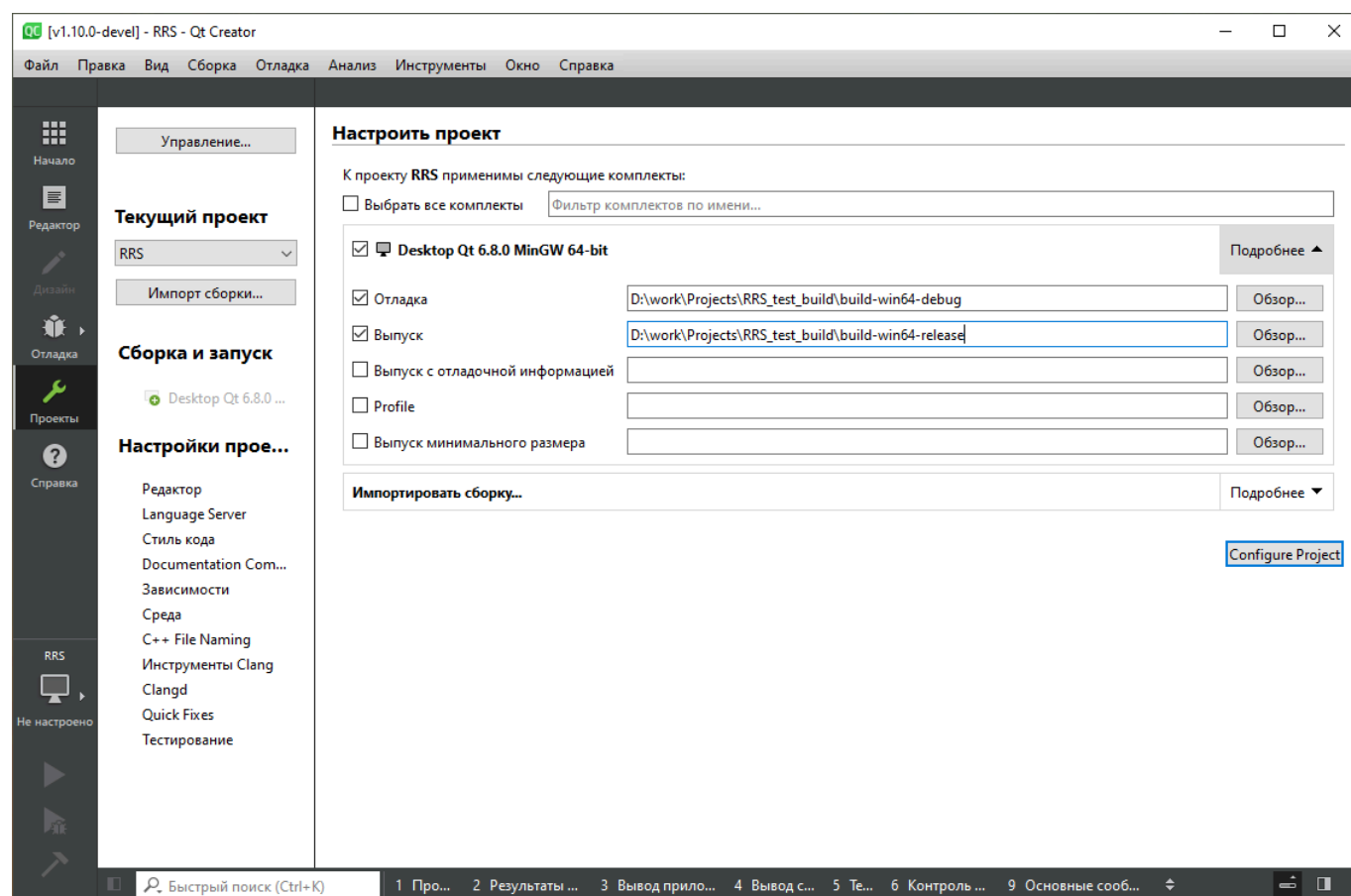
Запускаем IDE QtCreator из комплекта фреймворка Qt и открываем проект



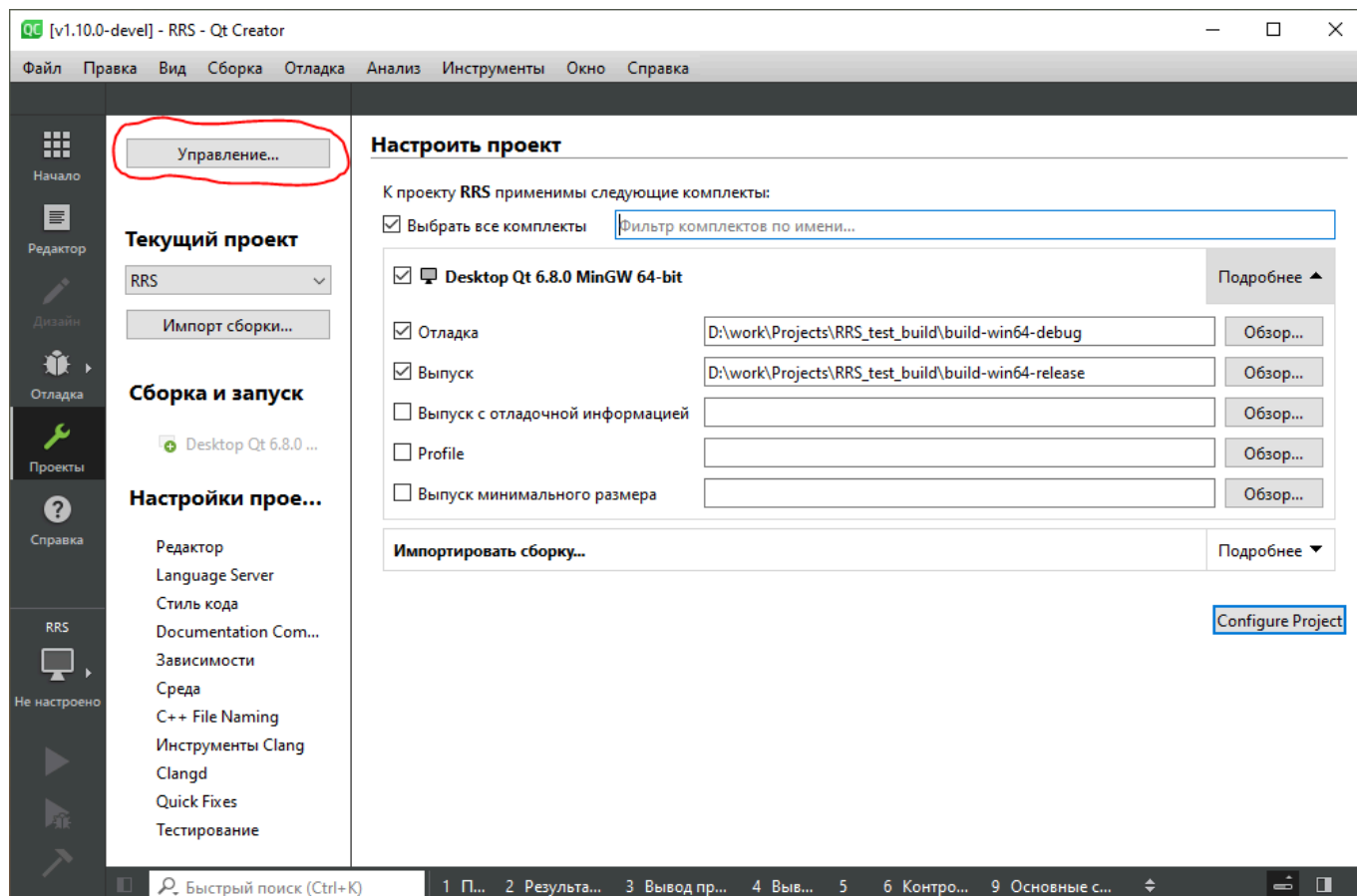
Указываем путь к скрипту сборки проекта



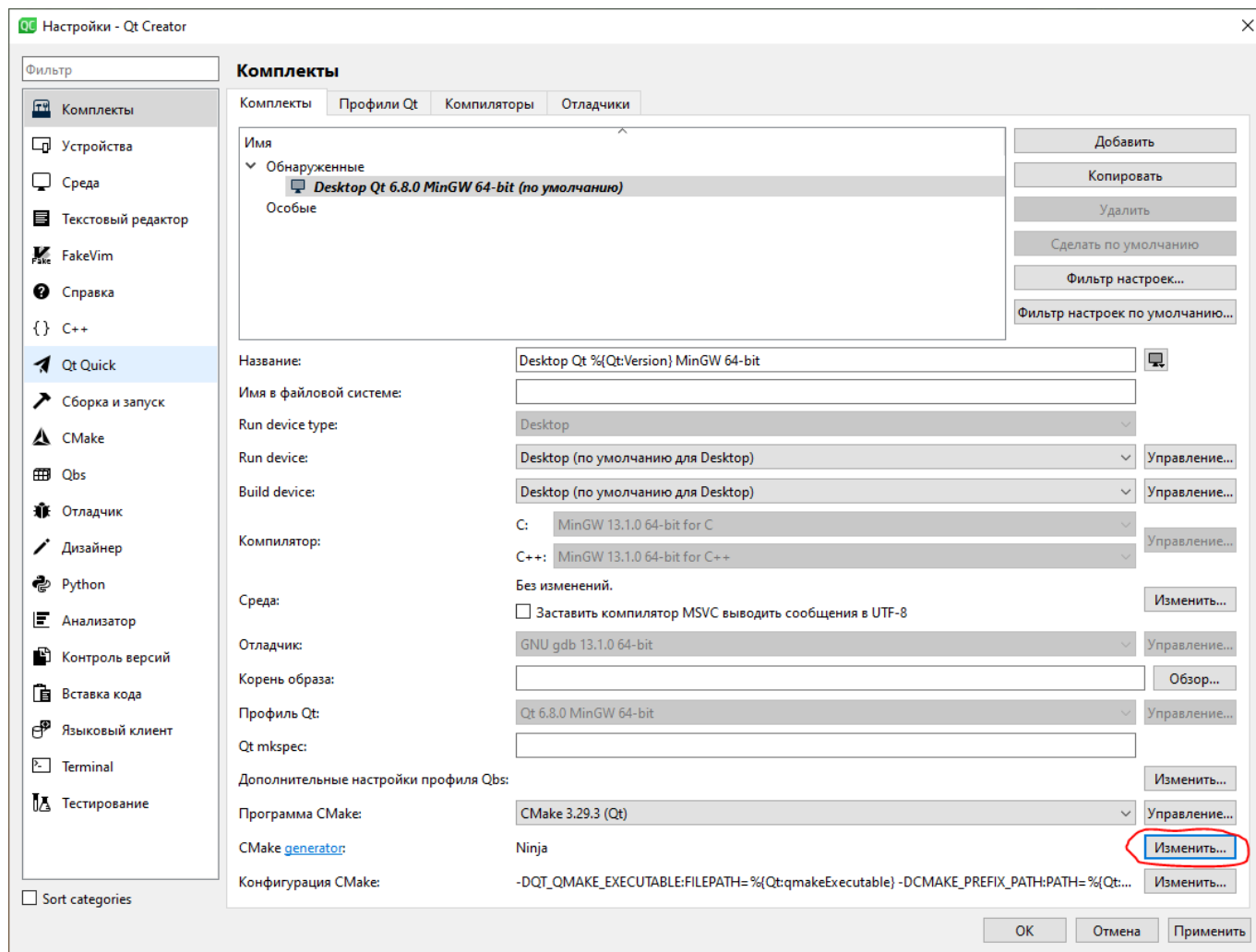
Настраиваем сборочные каталоги, аналогично тому, как показано на рисунке (с точностью до конкретного пути к проекту)



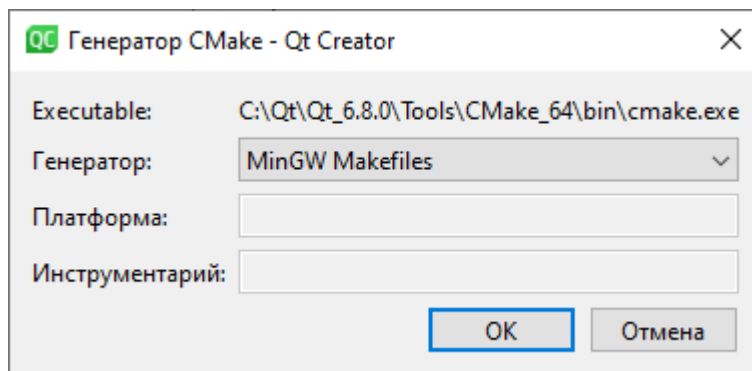
Кроме того, необходимо настроить CMake-генератор, для гарантированного использования экосистемы компилятора MinGW. Жмем кнопку [Управление](#)



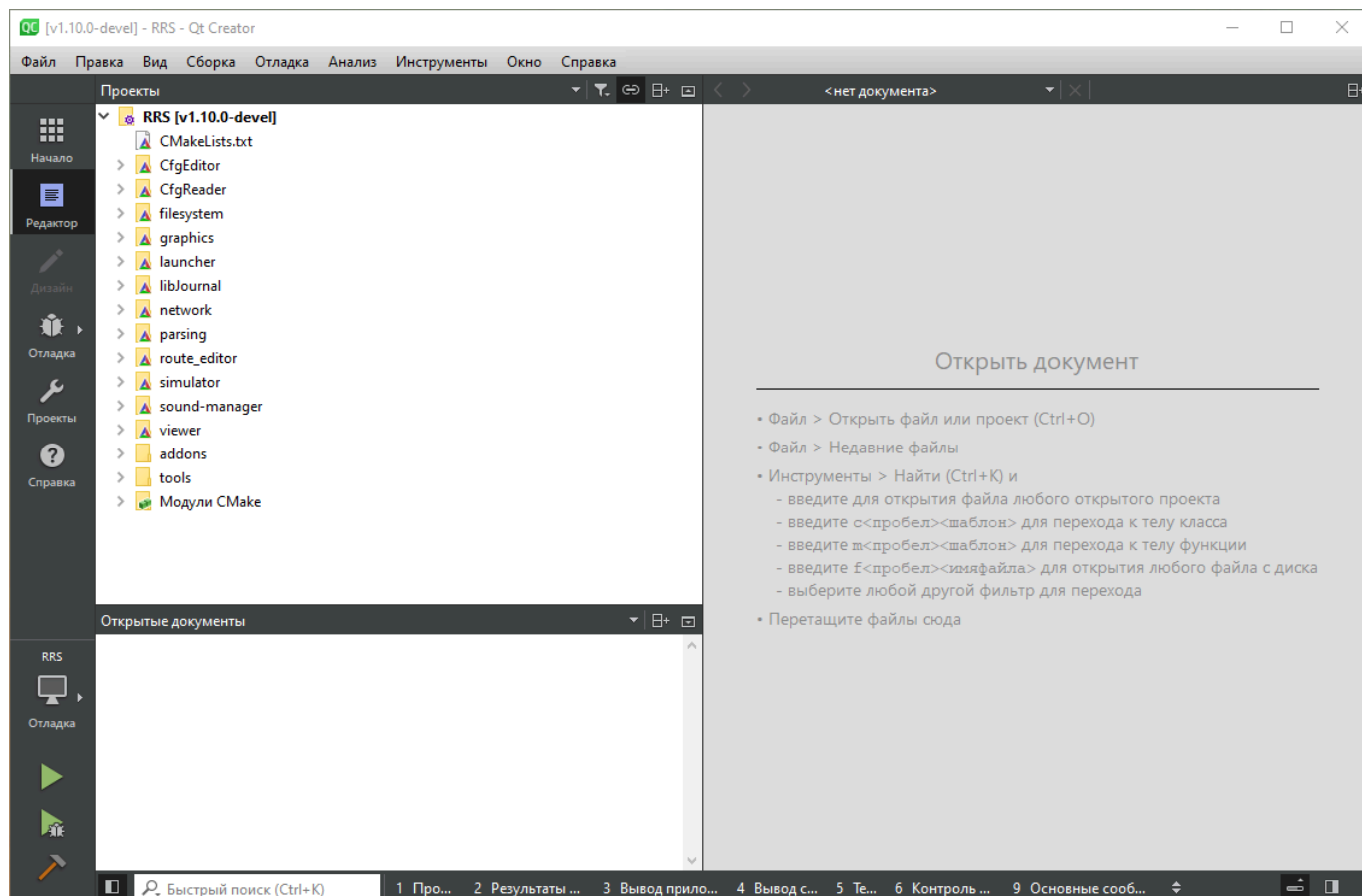
В появившемся окне настроек выбираем текущий комплект инструментов сборки и жмем кнопку **Изменить** напротив настройки CMake generator



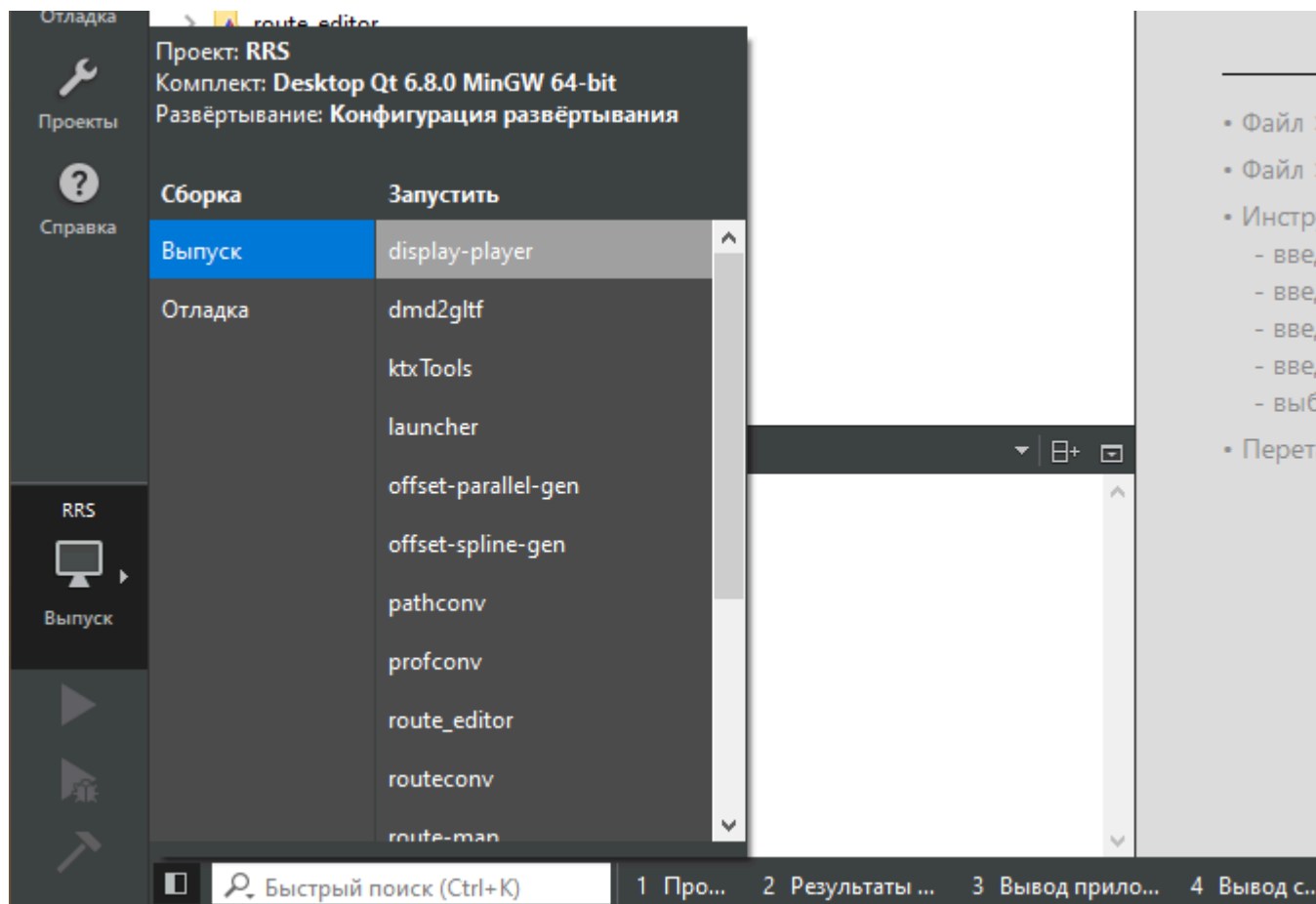
В появившемся диалоге выбираем генератор MinGW Makefiles



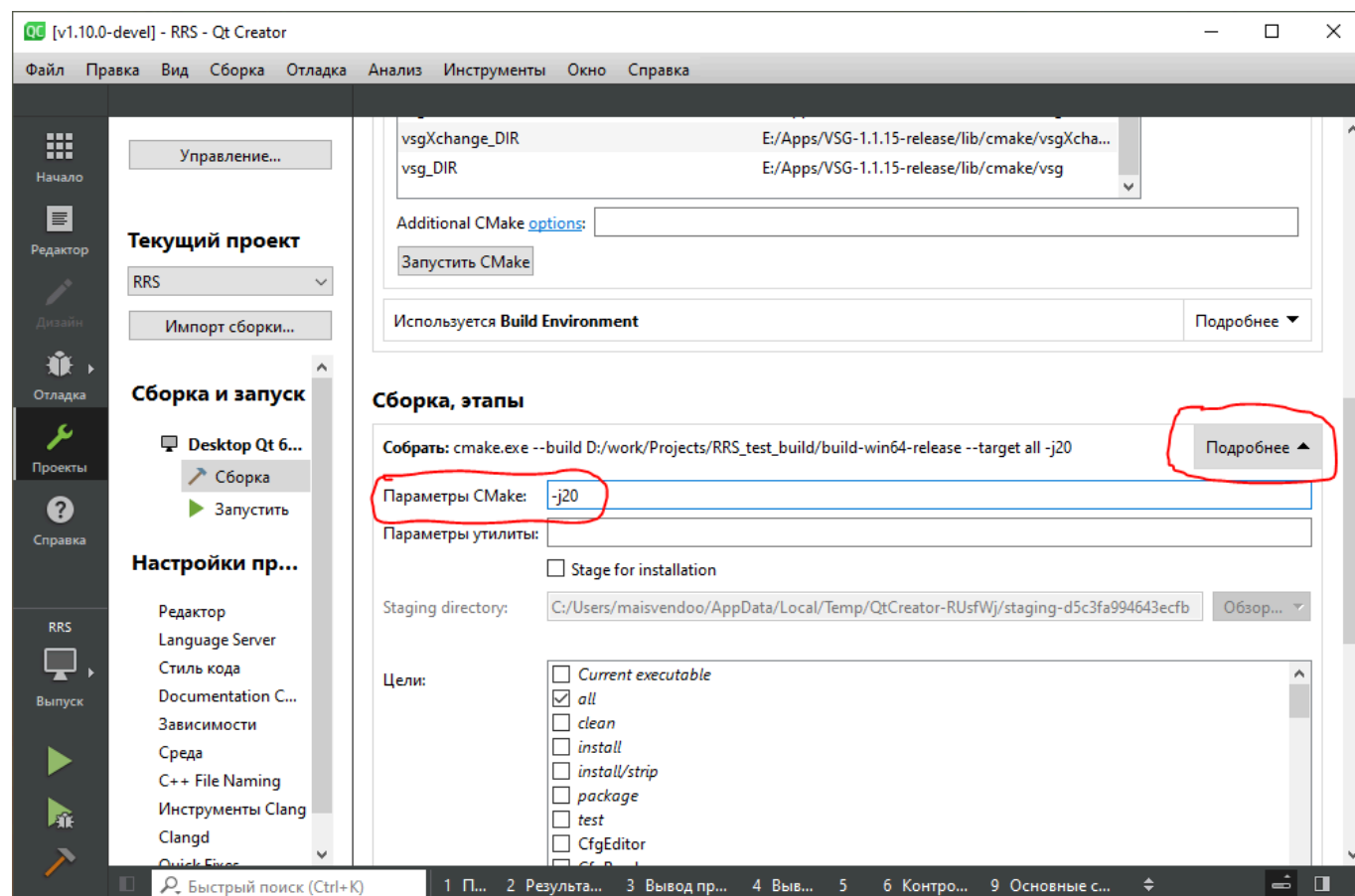
Жмем **OK** в этом и предыдущем окне, чтобы применить настройки и выйти. Далее нажимаем кнопку **Configure Project**. Если вы установили все зависимости и правильно настроили переменные окружения, вы получите готовый к работе проект, который выглядит так, как показано на рисунке



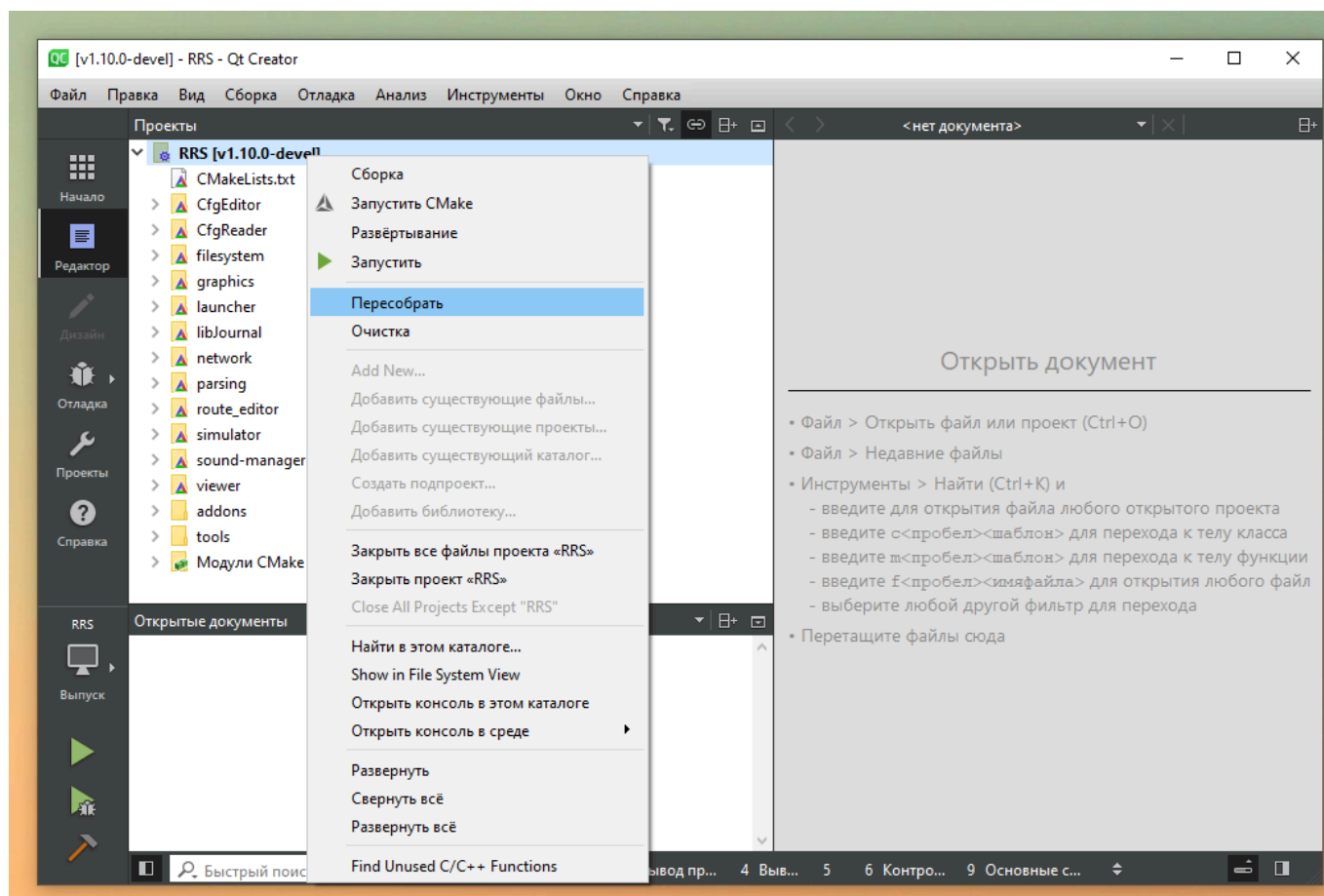
По умолчанию среда разработки настраивает отладочную сборку. Для настройки релизной сборки переключите вариант сборки с "Отладка" на "Выпуск", как показано на рисунке ниже



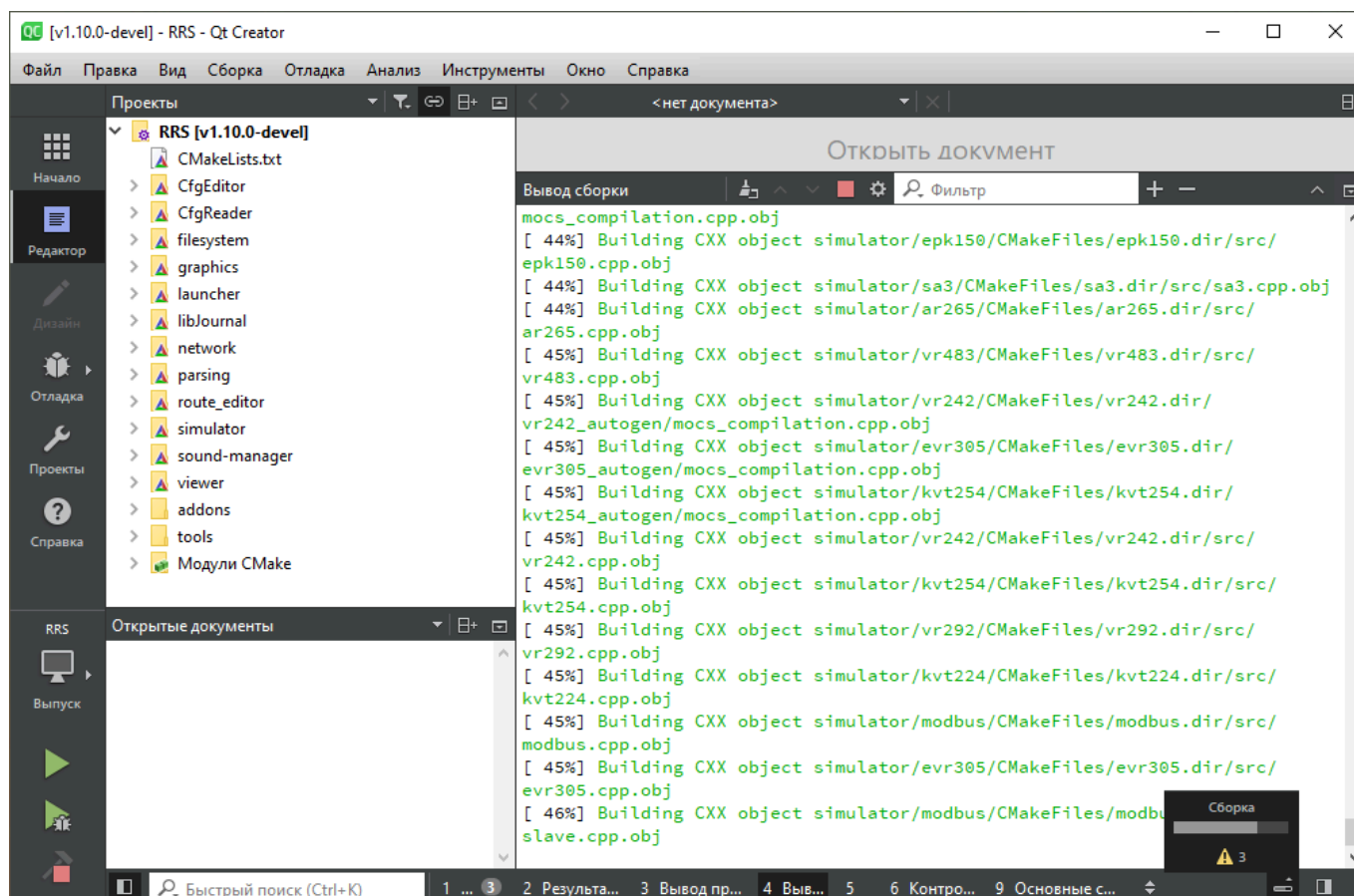
Большинство современных процессоров являются многоядерными и многопоточными. Для ускорения процесса сборки настоятельно рекомендуется задать много потоковую сборку. Через боковое меню **Проекты** -> **Сборка** -> **Сборка, этапы** -> **Параметры CMake** ключ для компилятора -jN, где N - число потоков вашего процессора



После этого правой кнопкой мыши в контекстном меню проекта выбираем пункт "Пересобрать" для полной очистки и сборки проекта.



Начнется процесс сборки, длительность которого определяется мощностью вашего процессора

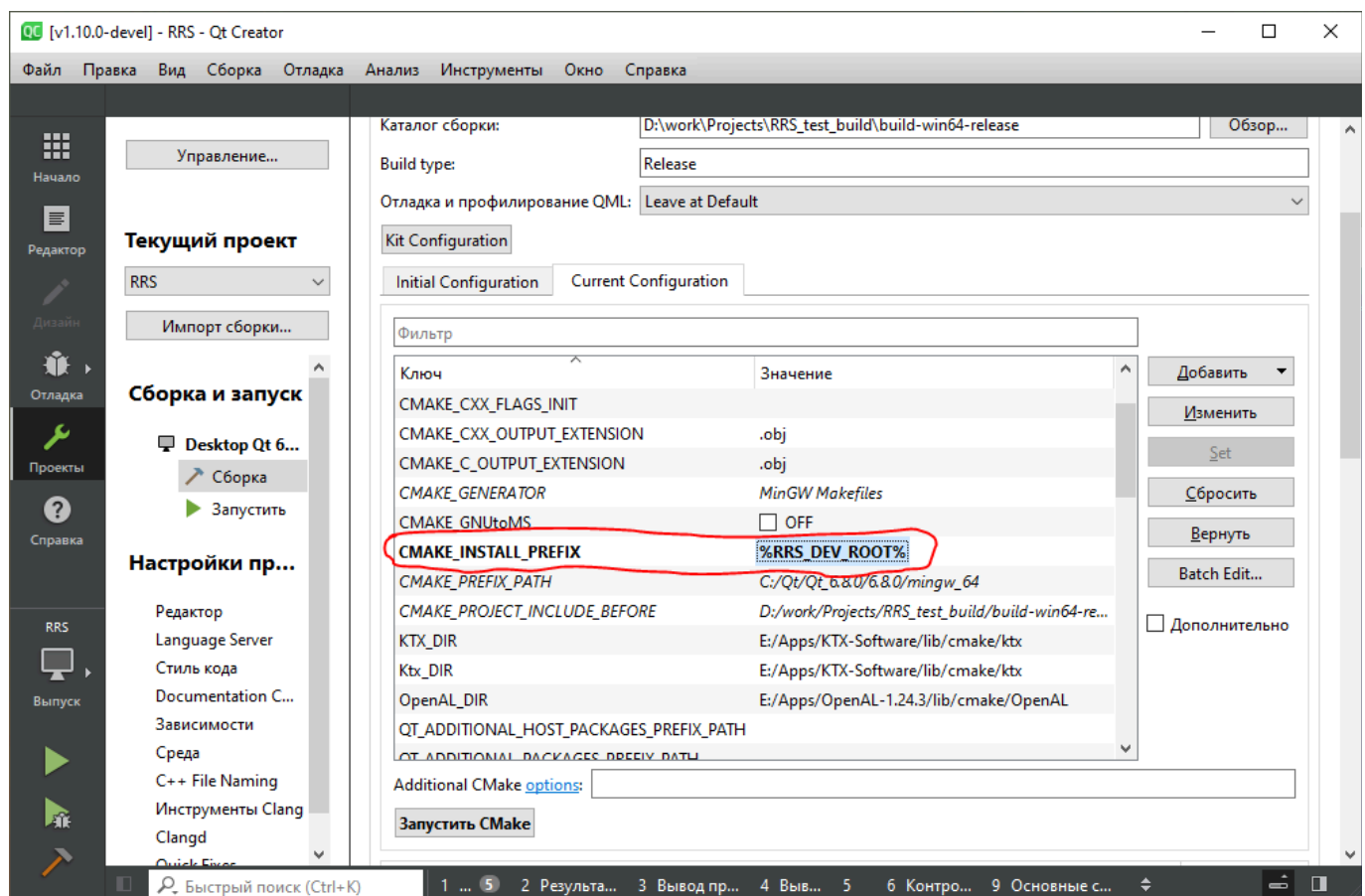


Внимание!!! Мы тщательно проверяем корректность сборки проекта с актуальными зависимостями. Правильность настройки сборки гарантирует сборку без ошибок! При получении ошибок сборки перепроверьте сделанные вами настройки. Если проблему не удалось устранить, обратитесь в поддержку на нашем техническом форуме.

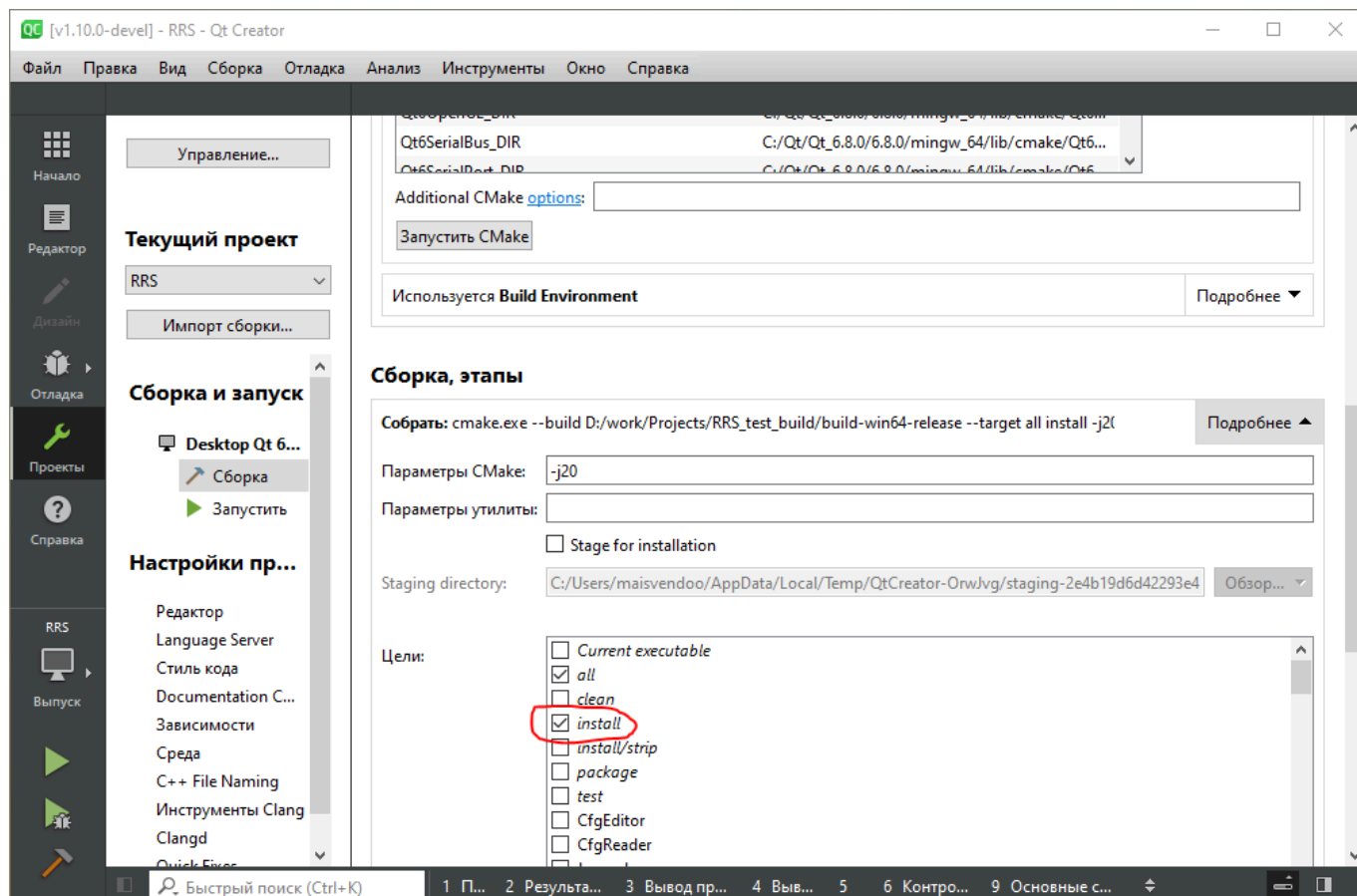
1.1.5 Развертывание собранного проекта

Для развертывания проекта, то есть получения работоспособной версии симулятора, необходимо выполнить следующие дополнительные настройки

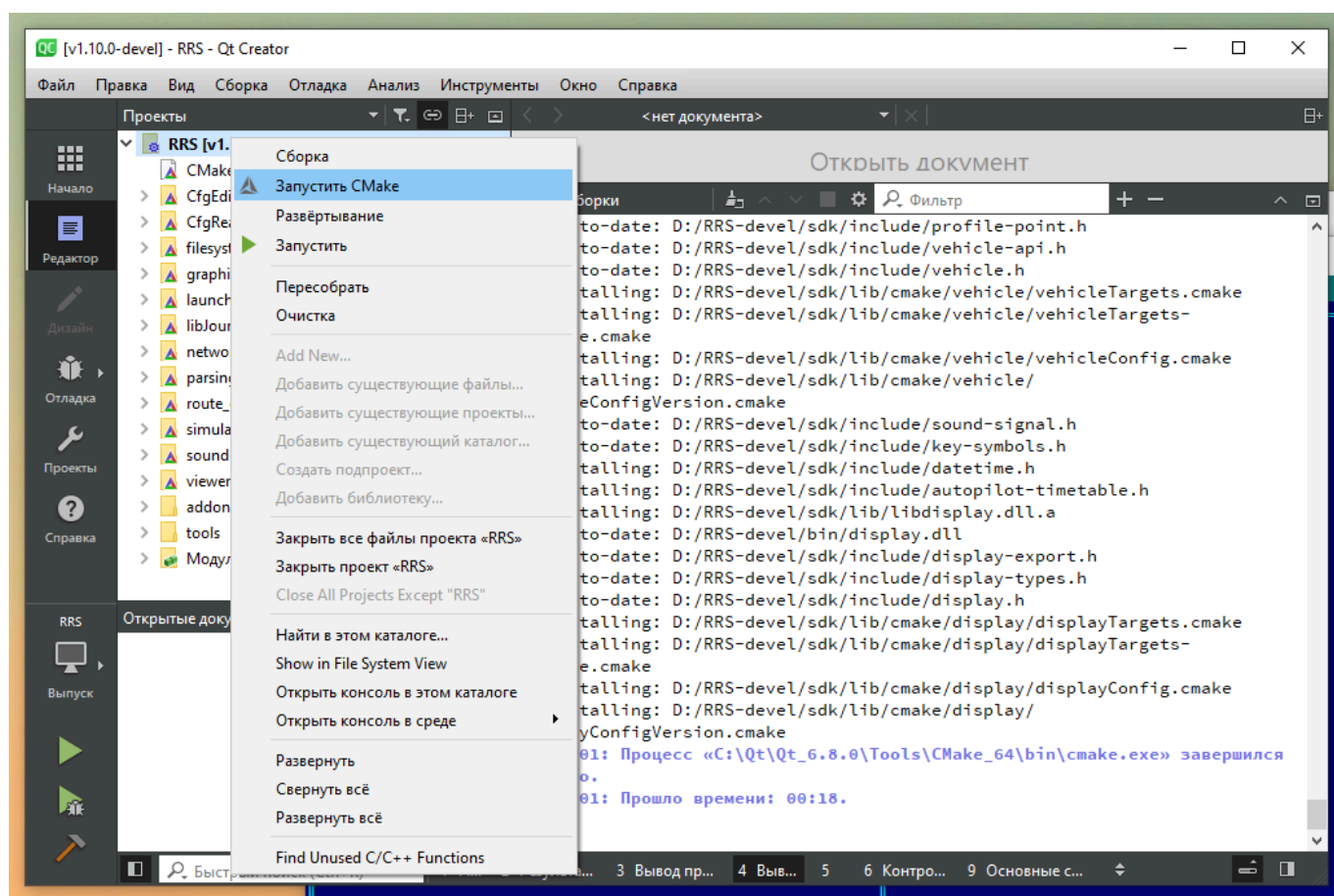
1. Установить переменную окружения `RRS_DEV_ROOT` равной пути к каталогу, куда будет скопирован симулятор, например `D:/RRS-devel`
2. В настройках проекта указать данную переменную в качестве значения для `CMAKE_INSTALL_PREFIX`



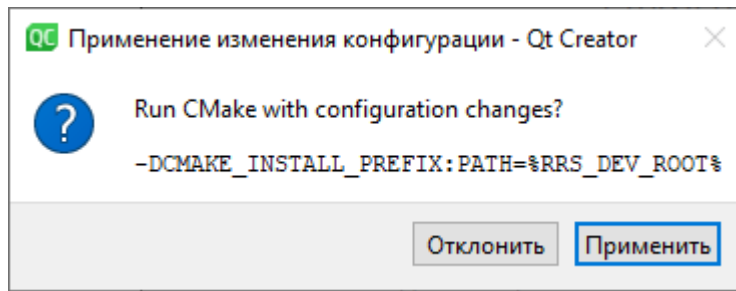
1. В параметрах сборки добавить цель сборки `install`



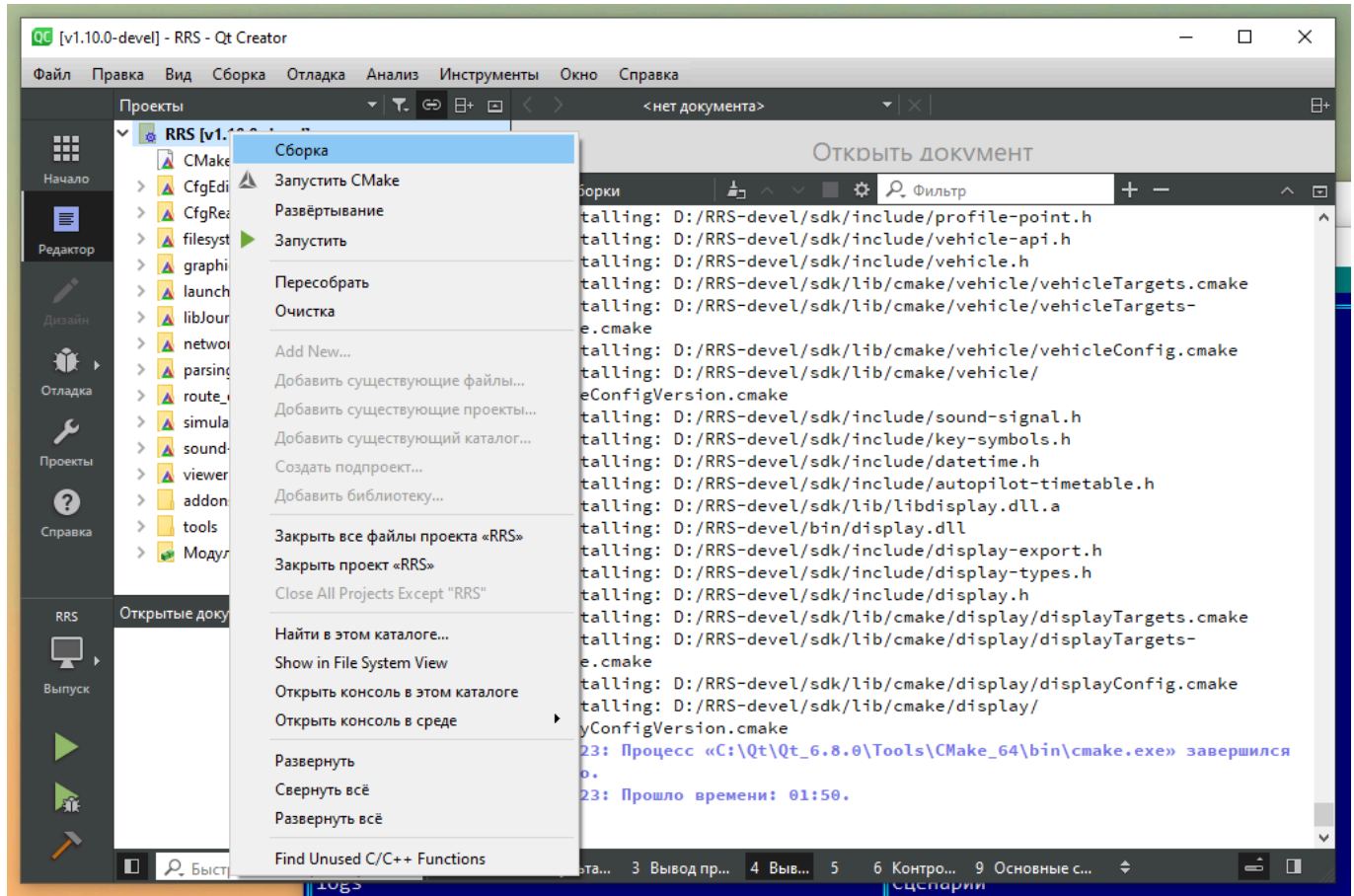
1. Выполнить повторный запуск CMake через контекстное меню в дереве проекта



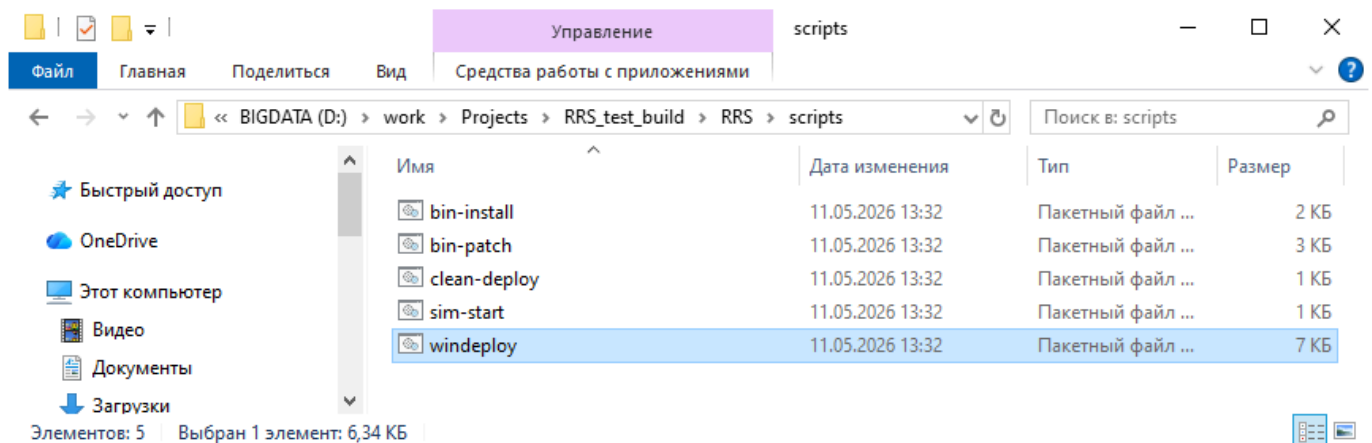
1. В появившемся диалоге принять изменение настроек переменных CMake



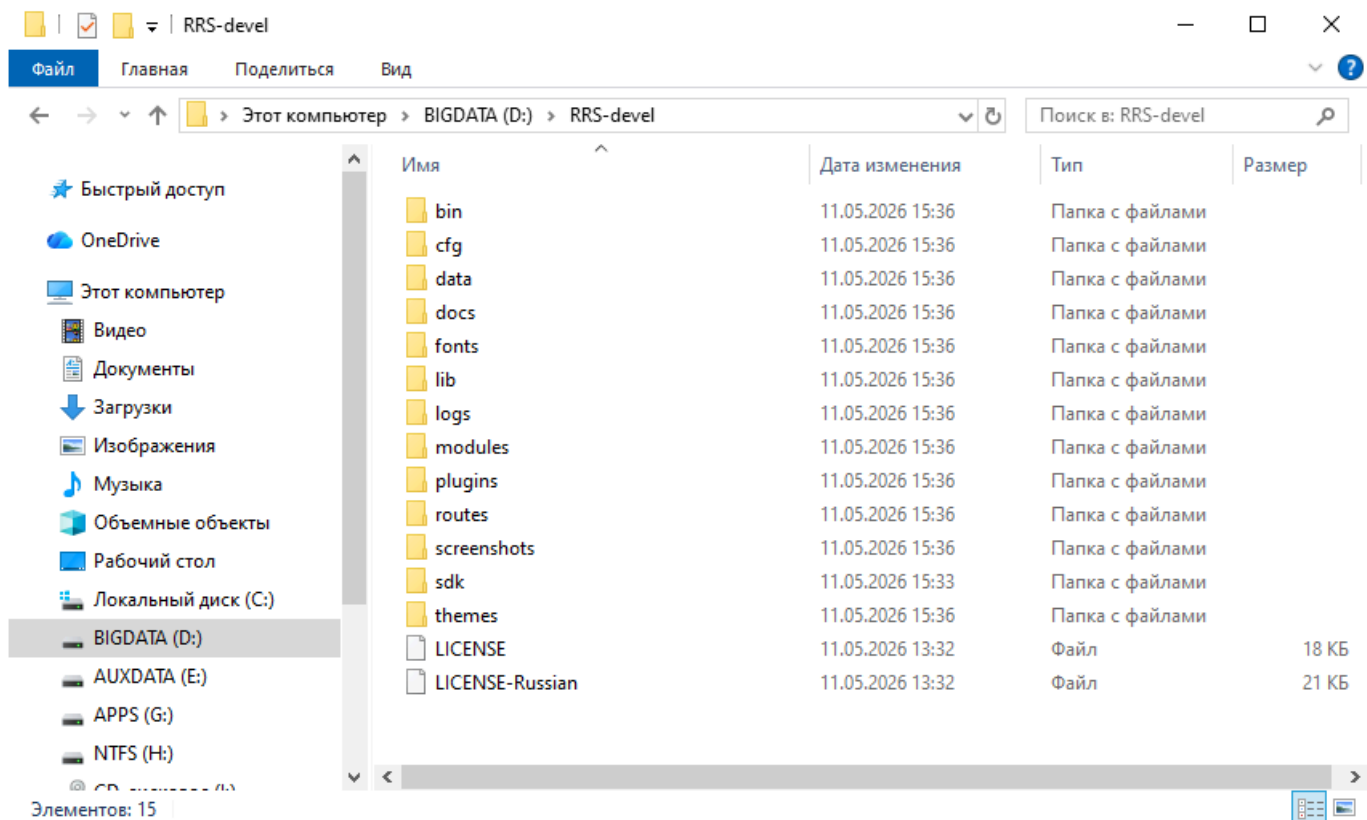
Выполните повторную сборку проекта



а после её окончания, в каталоге с исходниками запустите скрипт `scripts/windeploy.bat`



После его выполнения, получаем готовый к работе симулятор, в каталоге, указанном в переменной `RRS_DEV_ROOT`



1.2 Сборка RRS на macOS

Автор: Илья Крюков (@ikryukov). Инструкция актуальна для версии 1.9.1 и требует модификации под сборку 1.9.2, с учетом дополнительной зависимости KTX-Software. Текст будет обновлен по мере дальнейшей работы над проектом. Здесь и далее приводится авторский текст, с незначительным изменением форматирования.

Проверено: macOS 26.3, Apple Silicon (arm64), AppleClang 17.

1.2.1 Зависимости

Homebrew

```
brew install cmake qt@6 openal-soft sfml molten-vk \
    vulkan-headers vulkan-loader glslang assimp freetype
```

1.2.2 Сборка из исходников

```
DEPS=$HOME/RRS-deps
PREFIX=$DEPS/install
mkdir -p $DEPS && cd $DEPS
```

VulkanSceneGraph:

```
git clone https://github.com/vsg-dev/VulkanSceneGraph.git
cd VulkanSceneGraph
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PREFIX
cmake --build build -j$(sysctl -n hw.ncpu) && cmake --install build
cd $DEPS
```

vsgXchange:

```
git clone https://github.com/vsg-dev/vsgXchange.git
cd vsgXchange
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PREFIX -DCMAKE_PREFIX_PATH=$PREFIX
cmake --build build -j$(sysctl -n hw.ncpu) && cmake --install build
cd $DEPS
```

vsgImGui:

```
git clone https://github.com/maisvendoo/vsgImGui.git
cd vsgImGui
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PREFIX -DCMAKE_PREFIX_PATH=$PREFIX
cmake --build build -j$(sysctl -n hw.ncpu) && cmake --install build
cd $DEPS
```

Lua 5.4:

```
git clone https://github.com/maisvendoo/lua-cmake.git
cd lua-cmake
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PREFIX
cmake --build build -j$(sysctl -n hw.ncpu) && cmake --install build
cd $DEPS
```

sol2:

```
git clone https://github.com/maisvendoo/sol2.git
cd sol2
cmake -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_INSTALL_PREFIX=$PREFIX -DCMAKE_PREFIX_PATH=$PREFIX
cmake --build build -j$(sysctl -n hw.ncpu) && cmake --install build
```

1.2.3 Сборка RRS

```
cd /path/to/RRS
cmake -B build -DCMAKE_BUILD_TYPE=Release \
      -DCMAKE_PREFIX_PATH="$HOME/RRS-deps/install;/opt/homebrew/opt/openal-soft;/opt/homebrew"
cmake --build build -j$(sysctl -n hw.ncpu)
```

Результат: исполняемые файлы в `bin/`, библиотеки в `lib/`.

1.2.4 Запуск

```
cd bin/

# Терминал 1 — симулятор
./simulator --route experimental-polygon_v2.0 --scenario hello_scenario

# Терминал 2 — вьювер
VK_ICD_FILENAMES=$(brew --prefix molten-vk)/etc/vulkan/icd.d/MoltenVK_icd.json \
QT_QPA_PLATFORM=offscreen \
./viewer
```

`VK_ICD_FILENAMES` — путь к ICD MoltenVK (keg-only в Homebrew). `QT_QPA_PLATFORM=offscreen` — обход конфликта NSApp между Qt и VSG.